

DIP-CSE340 ASSIGNMENT 3

Identify the type of noise

Image1 has Pepper noise since it has isolated dark pixel regions.

Image2 has Gaussian noise since it has a grainy appearance with small intensity variations.

Method used for the image de-noising

Gaussian blurring was found effective for de-noising because it averages pixel values in a neighborhood, which smooths out noise variations.

This results in a reduction of pixel-by-pixel variation while preserving fine details like edges better as compared to other low pass filters, for e.g., box filter.

Pseudo-code of algorithm used for inpainting

Image_Inpainting describes the process to inpaint the patches and text in degraded image

Telea_Inpainting describes the algorithm used for inpainting

Algorithm Image_Inpainting(degraded_image):

Input: degraded_image

Step 1: Convert Image1 and Image2 to HSV

hsv_image1 $\leftarrow$ Convert degraded_image1 from BGR to HSV color space

hsv_image2 $\leftarrow$ Convert degraded_image2 from BGR to HSV color space

Step 2: Create Mask for Colored 'INPAINT' Text

Define HSV bounds for red text where (x1, y1) are coordinates of red text pixel in degraded_image1

lower_red $\leftarrow$ [hsv_image1[x1, y1][0] - 10, 100, 100]

upper_red $\leftarrow$ [hsv_image1[x1, y1][0] + 10, 255, 255]

red_text_mask ← Mask pixels in hsv_image that fall within lower_red and upper_red range

Define HSV bounds for green text where (x2, y2) are coordinates of green text pixel in degraded_image2

lower_green ← [hsv_image2[x2, y2][0] - 10, 50, 50]

upper_green ← [hsv_image2[x2, y2][0] + 10, 255, 255]

green_text_mask ← Mask pixels in hsv_image that fall within lower_green and upper_green range

Combine red and green masks for the 'INPAINT' text

text_mask ← Combine red_text_mask and green_text_mask using bitwise OR

Step 3: Create Mask for Black Patches

gray_image ← Convert degraded_image to grayscale

_, black_patch_mask ← Threshold gray_image using binary inverse thresholding

Step 4: Combine Masks

combined_mask ← Combine text_mask and black_patch_mask using bitwise OR

Step 5: Inpaint the Image

restored_image ← Inpaint degraded_image using combined_mask with Telea's Inpainting Method (cv2.inpaint() with flag cv2.INPAINT_TELEA)

Output: restored_image(inpainted)

Algorithm Telea_Inpainting(image, mask):

Input: image (input image with damaged/masked areas)

mask (binary mask where 1 = damaged, 0 = known region)

Initialize:

$D \leftarrow \infty$ for all pixels in image

for each pixel p in image:

if $\text{mask}[p] == 0$ then

$D[p] \leftarrow 0$ # Known region pixels have distance 0

if p is on the boundary of known region then

$\text{pq.push}(p, D[p])$ # Push boundary pixels into priority queue

While pq is not empty:

$p \leftarrow \text{pq.pop}()$ # Get the pixel with the smallest distance from queue

for each neighbor q of p :

if $\text{mask}[q] == 1$ then # If q is part of the unknown region

Update distance:

$D[q] \leftarrow D[p] + \text{distance_step}$ # Set distance for q

Update color/intensity:

$\text{image}[q] \leftarrow \text{weighted interpolation of known neighbors}$

Add q to pq with priority $D[q]$

Output: image

Explanation of the Pipeline

1. Convert Image to HSV Color Space:

- **Purpose:** The HSV color space allows for more effective color segmentation than the BGR color space, as it separates chromatic content (color) from intensity. This makes it easier to define ranges for specific colors.
- **Explanation:** The algorithm converts degraded_image from BGR (Blue, Green, Red) to HSV (Hue, Saturation, Value) using OpenCV's cv2.cvtColor function. In HSV, it's easier to isolate colors based on the Hue range, which is useful for targeting red and green text.

2. Create Mask for Colored 'INPAINT' Text:

- **Purpose:** Identify pixels that correspond to the "INPAINT" text in both images, which is either red (Image1) or green (Image2).
- **Explanation:**
 - Two sets of lower and upper HSV bounds are defined: one for red (lower_red, upper_red) and one for green (lower_green, upper_green).
 - These bounds are selected based on analysis of the images, targeting specific hue, saturation, and value ranges that cover the red and green colors present in the text.
 - OpenCV's cv2.inRange function creates a binary mask (red_text_mask for red and green_text_mask for green), where pixels within each HSV range are set to 255 (white) in the mask, and others are set to 0 (black).
 - Finally, the two masks (red_text_mask and green_text_mask) are combined with a bitwise OR operation to form a single text_mask. This mask will cover both red and green instances of the "INPAINT" text, ensuring compatibility for both images.

3. Create Mask for Black Patches:

- **Purpose:** Identify black regions in the image, which need to be inpainted to restore the underlying content.
- **Explanation:**
 - The image is converted to grayscale using OpenCV's cv2.cvtColor function to simplify thresholding, as we only need intensity information to detect black patches.
 - A binary inverse threshold (cv2.threshold) is applied with a low threshold value (e.g., 10) to capture very dark areas. Pixels below this intensity are set to 255 (white in the mask), while others are set to 0 (black).

- The resulting `black_patch_mask` highlights black regions, marking them for inpainting.

4. Combine Masks:

- **Purpose:** Combine both the text and black patch masks into a single mask for efficient inpainting.
- **Explanation:**
 - The `text_mask` and `black_patch_mask` are combined using a bitwise OR operation, resulting in `combined_mask`.
 - This `combined_mask` identifies all areas that need to be inpainted, including both the colored text (INPAINT) and black patches, allowing a single inpainting operation to handle all problematic areas.

5. Inpaint the Image:

- **Purpose:** Fill in the damaged or unwanted regions using known neighboring pixels.
- **Explanation:**
 - OpenCV's inpainting function (`cv2.inpaint`) is used with the `combined_mask` to restore the regions marked as damaged or unwanted.
 - The inpainting method selected is Telea's method (`cv2.INPAINT_TELEA`), which uses a Fast Marching Method (FMM) to propagate pixel values from surrounding known areas into the masked areas, smoothly filling them in.
 - Telea's method prioritizes pixels closer to the boundary, interpolating colors from neighboring pixels. This approach is effective for small, isolated regions (like text and patches) as it provides a visually consistent restoration.

6. Denoise the Image:

- **Purpose:** Reduce noise and smooth the inpainted areas to improve visual quality.
- **Explanation:**
 - After inpainting, some minor artifacts or noise may remain, especially around the edges of inpainted regions. Gaussian blurring (`cv2.Gaussian Blur`) is used to address this, which reduces noise while preserving edge detail.
 - This method computes the weighted average of pixels within a neighborhood around each pixel with weight decreasing as distance between pixels increases.
 - The result is a refined, denoised image that appears more natural and coherent.

(a) Image 1: Merit of the Selected Inpainting Algorithm

The selected inpainting algorithm works well for filling in smaller and less complex areas such as the sky or water, where there is a consistent texture and color gradient. The algorithm successfully removes text like "INPAINT" because it can easily replicate the blue sky texture without noticeable artifacts.

Explanation:

- The texture synthesis aspect of the inpainting algorithm helps to restore the sky and water regions seamlessly. These regions provide consistent, simple patterns that are easier for the algorithm to replicate without introducing noise or blur.
- The algorithm uses neighboring pixel values to interpolate and predict the missing areas, making it ideal for smooth and homogenous regions.

(b) Image 2: Demerit of the Selected Inpainting Algorithm

The selected inpainting algorithm struggles with larger, more complex structures, like the tree and the intricate texture of the landscape. When attempting to inpaint over the large object (tree in this case), the algorithm fails to recreate the fine details of the object, leaving the restored area looking unnatural and sometimes introducing visual artifacts or blurred regions.

Explanation:

- Patch-based algorithms or texture synthesis methods have limitations when dealing with large, highly detailed objects. They tend to perform poorly when they have to replicate intricate details like the branches of the tree or fine structures in the foreground.
- Since the algorithm relies on nearby pixels for filling in missing regions, it struggles with complex textures that vary significantly from their surroundings, leading to unnatural-looking results and reduced visual quality.

Improving the quality of output as progressing

- **Initial Issue:** The color of "INPAINT" text varied across image1 and image2, which made it challenging to define a single HSV range that worked for both Image1 and Image2.
- **Solution:** Instead of applying the HSV mask once, I performed iterative adjustments, slightly tweaking the HSV bounds based on feedback from each inpainting attempt. I also experimented with combining multiple HSV ranges to ensure that both red and green texts were captured simultaneously.
- **Result:** This iterative approach allowed me to fine-tune the HSV ranges progressively, ensuring that both colors of text were effectively removed without impacting other areas of the images.